

Cloud Uptime Archive: Open-Access Availability Data of Web, Cloud, and Gaming Services

Sacheendra Talluri , Dante Niewenhuis , Xiaoyu Chu , Jakob Kyselica , Mehmet Cetin ,
Alexander Balgavy , and Alexandru Iosup , *Member, IEEE*

Abstract—Cloud services are critical to society. However, their reliability is poorly understood. Towards solving the problem, we propose a standard repository for cloud uptime data. We populate this repository with the data we collect containing failure reports from users and operators of cloud services, web services, and online games. The multiple vantage points help reduce bias from individual users and operators. We compare our new data to existing failure data from the Failure Trace Archive and the Google cluster trace. We analyze the MTBF and MTTR, time patterns, failure severity, user-reported symptoms, and operator-reported symptoms of failures in the data we collect. We observe that high-level user facing services fail less often than low-level infrastructure services, likely due to them using fault-tolerance techniques. We use simulation-based experiments to demonstrate the impact of different failure traces on the performance of checkpointing and retry mechanisms.

Index Terms—Availability, checkpointing, cloud, failure, online games, simulation, web.

I. INTRODUCTION

CLoud services are critical to societal processes such as governance, healthcare, finance, science, socialization, and entertainment. However, we have little public data on the reliability of these services. In contrast, our community can access much public data on hardware failures [1], [2], failures in datacenter (internal) workloads [3], [4], software bug reports [5], and other paradigms such as HPC and networks of desktops [6]. To address this problem, we propose collecting *service* failure data from different kinds of services and different vantage points. In this work, we collect, analyze, and release the data and our tooling.

Cloud services, which are compute services accessed over the Internet, are now embedded into every societal process and have immense impact on society. This impact is similar to that cars had in the 20th century, when they democratized transport. However, cloud services have no standard reliability evaluation, unlike cars, which have institutions like the NHTSA [7] and

the NTSB [8] to evaluate their reliability. This makes it difficult for users to evaluate and learn lessons from critical failures of cloud services. A service failure can cause commercial operations to halt [9]. Cloud-service failures can also have a social impact, such as when students cannot attend classes [10], public transport is stalled [11], and emergency services stop being available [12].

The community has proposed several fault-tolerance mechanisms to overcome cloud failures. However, these mechanisms are usually evaluated against simple failure models [13], [14], [15], or failure traces collected from a single system [16], [17]. Often, the traces used are not public [18], [19]. The limited evaluation makes it difficult for researchers and practitioners to evaluate fault tolerance mechanisms comparatively.

We propose the Cloud Uptime Archive (CUA) to remedy the problem. The archive consists of availability traces from different cloud services, and tools to collect and analyze them. The trace archive approach has been successfully used multiple times already to the community's benefit. Example of trace archives include parallel workloads archive [24], grid workloads archive [25], failure trace archive [6], cloud software bugs [5], and workflow trace archive [26]. Closest to this work, the Failure Trace Archive was an early effort in collecting and analyzing computer systems failure traces, focusing on hardware failure traces and workstation availability data. In contrast, the data we collect is about cloud services. Cloud services have multiple software layers between the user and the hardware, making service failure qualitatively and quantitatively different from hardware failure.

As an example of the data we collect and the analysis it enables, we analyze failures of the social network service X/Twitter, before and after it was taken private. Fig. 1 depicts Twitter's failures collected from the crowdsourced failure reporting website Outage.Report. For the year up to the date Twitter was taken private (28 October 2021 - 2022), Twitter experienced 23 failures with 1,084 user-reports. In the year starting from the day it was taken private (29 October 2022 - 2023), Twitter experienced 13 failures with 199 user-reports. The low number of failures indicates that a large web service can keep operating even after a massive reduction in headcount. However, failures alone do not portray the full picture. We combine failure and user-count data (also depicted in Fig. 1) and analyze this further in Section II.

Existing work [27], [28] studies failures reported by their cloud operator. That is not enough, as failure reporting is a

Received 29 January 2025; revised 14 January 2026; accepted 2 March 2026. Date of publication 12 March 2026; date of current version 27 March 2026. This work was supported in part by EU Horizon Graph Massivizer and EU MSCA CloudStars Projects and in part by the National Growth Fund through the Dutch 6G flagship Project "Future Network Services". Recommended for acceptance by E. Smirni. (Corresponding author: Sacheendra Talluri.)

The authors are with Vrije Universiteit Amsterdam, 1081 HV Amsterdam, The Netherlands (e-mail: sacheendra.t@gmail.com).

We release the data, and the analysis and simulation tools, as open-source artifacts available at <https://github.com/atlarge-research/cloud-uptime-archive>.

Digital Object Identifier 10.1109/TPDS.2026.3673519

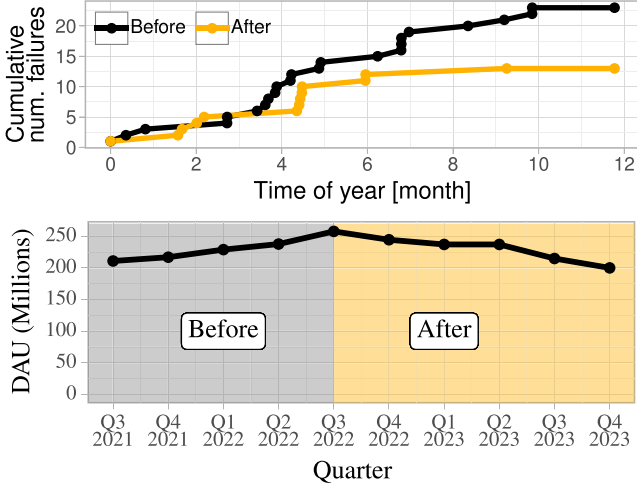


Fig. 1. X/Twitter failures and daily active users (DAU) before and after it was taken private. DAU data estimated by Statista [20], [21] and Sensor Tower [22] via NBC [23].

subjective process, and the human and economic incentives involved mean that just studying one source of failure information can distort our view of failures. The distortion of focusing on just one source for experiment traces has happened before with workflow traces [29]. Therefore, to provide a different viewpoint, we collect failure data from multiple aggregators of user-reported failures. For yet another vantage point, we also collect active user counts of online services when possible.

Collecting and characterizing failure reports of cloud services is challenging. First, the data collection is hampered by the lack of persistence in data published by crowdsourced aggregators; most of the data remains available for less than a day. Second, analysis of provider-reported failure data is hampered by inconsistent reporting in natural language. Third, data from different sources comes in different forms and is not directly available as failures. Extracting failures from user reports and active user counts is challenging.

Towards understanding the reliability of cloud services and standardizing the evaluation of fault tolerance mechanisms, we make the following contributions:

- 1) We collect long-term failure data from different types of cloud services, Web, Infrastructure Cloud, and Online Games. Apart from operators' status pages, we collect data from unique vantage points such as crowdsourced failure reporting websites and player counts for online games. We describe the nature of failures, as reported by operators and users, in Section II, and compare failures from different sources to validate the dataset in Section IV.
- 2) We analyze the MTBF and MTTR, time patterns, and failure severity using statistical techniques in Sections V to VII. We compare the results of our statistical analysis to existing datasets such as the FTA [6] and Google cluster traces [30].
- 3) We manually analyze the natural language reports of a one-year-long subset of data, and the rest using LLMs, to identify failure symptoms and root causes in Section VIII.

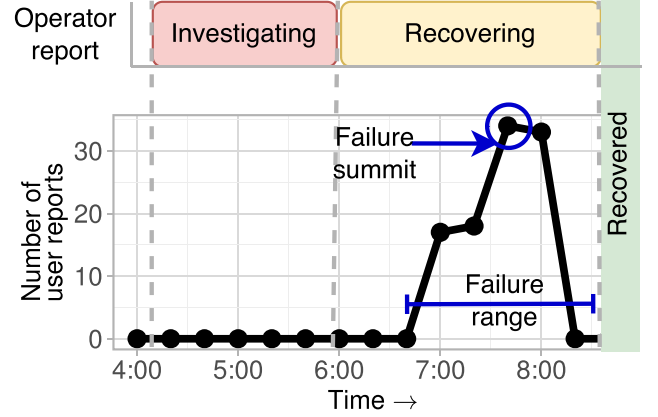


Fig. 2. User and operator reports of the Github failure on July 13, 2020. The failure range and failure summit are highlighted.

- 4) We use traces from our dataset to demonstrate how it can be used to evaluate fault tolerance mechanisms. We evaluate variants of fault-tolerance mechanisms such as checkpointing and retries using simulation in Sections IX and X.
- 5) We release the dataset and the tools we use to collect, extract, clean, and analyze the data as FAIR Open Science artifacts. **The dataset covers 27 data sources** and covers popular services such as infrastructure clouds (AWS, GCP, Azure), infrastructure services (Github, Atlassian), social media (Instagram, Twitter), and online games (Minecraft, Runescape). The dataset and tools are available at <https://github.com/atlarge-research/cloud-uptime-archive>. The dataset is also available on Zenodo at <https://doi.org/10.5281/zenodo.14712441>.

II. ANATOMY OF A FAILURE

We define failure as when a cloud service cannot provide all or some of its functionality to all or some of its users. A system's true state is only visible to the service operators, and for sufficiently complex systems, even they might not have full visibility [31]. However, different actors, such as operators, users, and metrics collection systems, can measure the system's state and, hence, the failure state from their own perspective. These different measurement apparatuses are the *vantage points* from which we can analyze a system's failure performance. We describe two examples to demonstrate why the different vantage points are necessary.

For each failure, users report their experience of the failure to a third-party reporting service like DownDetector or Outage.Report. An exemplary sample of such reports for a failure experienced by the service Github is depicted in Fig. 2. We identify all adjacent timestamps with non-zero reports as a *failure range*. The maximum number of reports for a single timestamp in that range is the *failure summit*. The operators also label the failure with different statuses at different times, as depicted in Fig. 2.

Short-term dynamics (Github example): Fig. 2 depicts the user reports of a failure experienced by the popular code-hosting

service on July 13, 2020. The figure depicts user reports and the status reported by the operator. Github reported this incident as a major failure [32], and it was covered by news media [33]. The operator noticed the failure and started investigating at 4:06 AM UTC. However, no users reported the failures as it was early in Europe and late evening in the USA, where most Github users are from. The operator identified the cause of failure at 5:53 AM UTC and initiated recovery. Users started using the service in the morning and reported the failure to the third-party website. The number of reports increased as the morning progressed. As the operator reached the end of the recovery process, the number of user reports also dropped.

We observe a time lag between when the operator reported the failure and when users observed it. This time gap allows operators to recover from failure without the users even noticing. Systems developers can use the two different failure traces, from operators and users, to evaluate their fault-tolerance mechanisms.

Long-term dynamics (X/Twitter example): Twitter's status page was taken down since twitter was taken private. The exact uptime has not been officially reported or known since. Since being taken private, up to 75% of Twitter's staff were let go [34]. Whether the service availability has decreased, as an expected consequence of the layoff, is unknown. We perform a preliminary investigation towards answering this. We compare Twitter's uptime from when it was taken private to the year after in Fig. 1. For the year up to the date Twitter was taken private (28 October 2021 - 2022), Twitter experienced 23 failures with 1,084 user-reports. In the year starting from the day it was taken private (29 October 2022 - 2023), Twitter experienced 13 failures with 199 user-reports. The low number of failures indicates that a large web service can keep operating even after a massive reduction in headcount.

We also investigate if the low number of failures is due to less people using twitter. We infer daily active user data (DAU) by combining the DAU data up to 2022 from Statista [20], [21] with the user decrease data for the subsequent year from Sensor Tower [22] via NBC [23]. The DAU data indicates that while Twitter's DAU has decreased, it has not decreased to the extent failures have decreased. The reduction in failures without a significant reduction in DAU indicates that the headcount reduction at Twitter has not had a significant impact on service reliability.

III. METHOD FOR COLLECTION AND INTERPRETATION OF CLOUD UPTIME DATA

We aim to collect longitudinal availability data for the main ICT services for 21st-century economies and societies. We collect availability data for different cloud services from different vantage points: service operator status pages, user-reports to failure reporting websites like Downtdetector, and user counters. The different vantage points each have their biases. By collecting data from all three of them, we expect to get a fuller picture.

We collect data from different services intermittently over a 17 a period (2007-2024). We collect data from web, cloud, and online gaming services. These categories are representative of what our society finds most lucrative and popular.

We extract failure ranges and peaks from the user reports. We describe the process in Section III-B. We extract failure severity based on number of affected services and number of user reports in Section III-D.

A. Data Collection

We use a web crawler which retrieves raw data from different sources. The different sources are summarized in Table I. We collect data every hour for some sources and every 6 hours for others, based on their update frequency and throttling. For example, we collect data every hour from outage report even though the failure cause data is reset every 20 minutes, to avoid throttling. We then parse the raw data, which is in HTML or JSON format. We had to design and implement custom parsers for each data source to extract the required information.

One obstacle we faced while collecting user reports of cloud failures was that failure reporting websites like Downtdetector blocked and throttled our data collection efforts. We were able to obtain historical data using distributed household proxies. However, even these effort were successfully blocked. Hence, that dataset ends in December 2020.

B. Failure Extraction

Some data sources, such as operator status pages, provide a clear failure period with a start time and an end time. But other crowdsourced sources only provide number of user reports of failures (Outage Report and Downtdetector) or number of online users (Runescape and Minecraft). To extract failure data from these crowdsourced sources, we detrend the data and then use peak extraction. Our hypothesis is that failures are periods with a sudden increase in the number of user reports of failures or a sudden decrease in the number of users.

For preprocessing, we check whether there is a seasonality, meaning a recurring pattern with a fixed period. If such a pattern exists in the data, it means that the fluctuations caused by that pattern are not anomalous. To facilitate finding the points which fall outside of this regular pattern, it is useful to remove the seasonal component from the data. The data may contain a long-term trend, which we also remove.

To extract and remove these two components from the data, we use STL [35]. STL decomposes the time series into the trend component, the seasonal component, and the remainder. This remainder is useful for identifying anomalies as it contains deviations from the regular patterns. We chose STL because it supports arbitrary sampling frequencies and because its simplicity allows for fast computation; something which is rather important given the high-frequency data which, in the case of Runescape, has over 2 million data points.

We employ two methods for peak detection. One to identify peak location and another to identify the duration of the peak. Both methods are depicted in Fig. 3. This combination of location and duration correspond to a failure.

The first method (illustrated in the left subfigure) calculates the change in the number of user reports or users over a rolling window (W), operating on the deseasonalized dataset. When a data point is N standard deviations below the mean of the window, it is marked as an anomalous point. The magenta line

TABLE I
SUMMARY OF COLLECTED DATASETS

ID	Dataset Name	Time Period	Services Tracked	Fields
WU	Web User Reports (Outage Report)	April 2019 - September 2024	Apple Servers, Facebook, FB Messenger, GitHub, Gmail, Instagram, Netflix, Snapchat, Skype, Twitter, WhatsApp, YouTube	Time, number of reports, reason
CU	Cloud User Reports (Downdetector)	January 2018 - December 2020	Google Cloud, AWS, Microsoft Azure	Time, number of reports
WO/CO	Web/Cloud Operator Reports	April 2019 - September 2024	Google Cloud, AWS, Azure, Slack, Github, Atlassian services, Discord	Time period, affected services, reason
G	Online Game Player Count	August 2020 - May 2021 (Minecraft) August 2007 - March 2015 (Runescape)	Runescape, Minecraft (Hypixel, The Hive) Cubecraft, Minehut	Time, number of online users

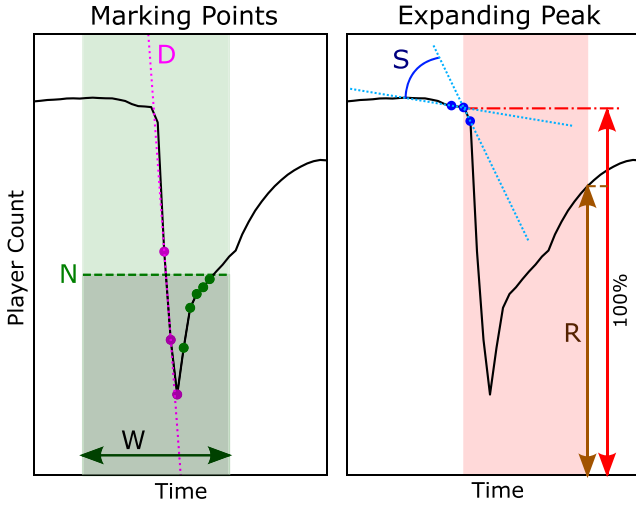


Fig. 3. Peak detection algorithms.

(D) represents the first-order difference threshold for marking points.

To identify failure duration, the anomalous points are expanded backwards and forwards in time. This is illustrated in the right subfigure. For each point marked as anomalous, its preceding points are evaluated one by one until a sharp drop in the first-order difference (S) on the raw value is observed. The peak is considered to have ended when a certain fraction of players (R) have been regained. The red arrow shows 100% of the original player count for reference.

C. Natural Language Report Data Extraction

Cloud services report failure information such as the root cause and affected services in natural language. Extracting key information from failure reports is time- and labor-consuming because the reports have idiosyncratic formatting and are written by different people.

We use two different approaches for this. First, we apply large language models (LLMs) to automatically extract high-confidence fields from failure reports. Second, we use manual human analysis for low-confidence fields to extract information from a one-year subset of the data.

To determine the confidence of a field, we extract a 10% subset into a test set. We manually extract the fields we need, such as the service name, service category, location, user symptoms, and

root cause for each report in the test set. Then, we use LLMs to extract the same information for the rest. We compare the results from the LLMs to our manual classification results. We consider high-confidence fields as those for which the LLMs have over 70% accuracy.

To prevent bias from a single LLM, we use three LLMs and have them vote on the correct answer. For the vast majority of failure reports, all three LLMs unanimously agree on the answers. The LLMs we use are: ChatGPT-4, Claude, and Gemini Pro. All of them were accessed through their respective APIs.

We use the method we describe above to study the failure reports from three cloud operators: AWS, Azure, and Google Cloud. We present the results of the analysis in Section VIII.

D. Interpreting Severity

The datasets we collected have different descriptions and interpretations of failure. But a unified representation is better for using the data to evaluate fault tolerance techniques. An important part of this is quantifying the severity of a failure. A failure's severity can be based on the number of people affected, the qualitative way they were affected, the number of services affected, and the duration.

We focus on unifying three different failure severity metrics into a unified format. One metric is the severity of the failure defined by the cloud operator. Cloud operators define this based on the number of people affected, the impact of the failure, and other considerations.

A second metric is the number of user reports of failure. We first extract failures from user report data using the peak detection technique from Section III-B. We then identify the 5th and 95th percentile of user reports for all extracted failures. We set any value above the 95th percentile as failure severity 1 and any value below the 5th percentile as failure severity 0. The values in the middle are interpreted to be between 0 and 1 using min-max scaling.

The third metric we use is the number of online users. We use the same method that we use for the user reports, except the direction of severity is reversed. The periods with the fewest amount of online users are failure severity 1 and the ones with the highest amount of users are severity 0, meaning no failure.

A convenient result of interpreting failure severity as lying between 0 and 1 is that we can use these as the probability of a service failing to respond in experimental evaluation. We use the failure severity value as the fraction of machines that failed

start time	end time	severity
------------	----------	----------

Fig. 4. CUA trace format.

in Section IX. We use it as the fraction of network requests that failed in a given time period in Section X.

E. Cloud Uptime Archive Data Format

We release both the raw failure data we collect and failure traces in a normalized format. The normalized traces are made for easy consumption by a real-world experiment or simulation. We describe the normalized trace format here. If users want granular control, they can always fall back to the raw data.

Each file in the dataset represents the failure trace for a single service, from a single vantage point. For example, Github (service) failures reported by users (vantage point). This information is incorporated into the filename, e.g., `cua_github_outagereport.csv`. Each file is in CSV format and contains multiple lines. Each line represents a failure. In the normalized format, failures do not overlap. However, in some raw data sources where each service provided by an operator is tracked independently, failures can overlap. We combine them into a single failure during normalization.

Fig. 4 depicts the format of each line in a trace file. The start time and end time represent the start and end of the failure. These times are not normalized to start at 0. The first failure does not necessarily occur at the start of the experiment. Users should normalize the timestamps in the trace to their desired start time. The final field is normalized severity as we described in Section III-D. It has a value between 0 and 1. 0 represents no failure and 1 represents total failure. This can be used as the probability that a machine or set of machines fail, as we do in Section IX. It can also be used as the probability that a request fails, as we do in Section X. The severity can also have other interpretations depending on the user's experiment.

IV. VALIDATION BY COMPARING DATA FROM DIFFERENT SOURCES

To assess the usefulness of user reports in identifying failures, ideally, we would compare them to the ground truth of failures for all services we monitor. However, this is not possible, as the operators rarely publicly report all the failures their cloud services experienced. Instead, we take as ground truth the set of official updates about failures currently reported on each cloud service's status page. We have such information from status pages of several operators such as Github, Atlassian, AWS, GCP, Azure, and more. This allows us to show evidence of whether the failures reported by users are similar to the failures the operator self-reports. Furthermore, this gives insight into whether the metrics cloud services are tracking are what users experience as failures or if users experience something different. Our main findings are:

O-1: There is overlap between the *major* failures on the operator status page and those reported by users.

O-2: There is little overlap between *all* the failures self-reported by the operator and crowd-reported by users.

Fig. 5 depicts Github and AWS failures as (officially) self-reported by the operator and as reported by users to Outage Report. The horizontal axis represents the days of the year. The first day of each month is labeled. The vertical axis represents the time of the day. A single box represents one hour of one day. There are 5 major issues officially reported by Github and 4 major issues reported by AWS. 4 out of the 5 major Github issues overlap user reports to Outage Report (O-1). 2 out of the 4 major AWS issues overlap issues reported by Outage Report. We further find 8 and 10 official minor failures that overlap with user reports for Github and AWS, respectively. This gives evidence the overlap cannot occur randomly, so *user-reported failures are consistent with the ground truth*.

We also observe that many official minor reports have *no* overlap (O-2). This suggests little overlap between what users and Github perceive as important enough to report. The difference between the sets of official and user reports stems from the different kinds of failures they track. Minor events reported by Github range from degraded performance to problems with forking and accessing the website. In contrast, the most popular reason for events reported to Outage Report by far is "Website Down", which accounts for about two-thirds of the reports for Github. In line with similar conjectures about software-developing companies [36], we conjecture the official reports correspond to failures tracked by Github's *internal* metrics, whereas user-reports correspond to events with high visibility for the (*external*) users.

The difference between officially reported and user-reported failures suggests a gap in the definition of failures that are to be reported. On the one hand, as researchers and expert users, we want detailed information when a service we use is not functioning. On the other hand, a service cannot publish thousands of small events as failures because this could fatigue the users and be misinterpreted as pervasive unreliability.

Our findings in this section motivate the need to revisit current failure-reporting methodologies, to make them identify and classify failures consistently with what the users experience. We recommend reporting both global failures and fine-grained detailed failures per operational instance (shard) of each service; for example, the Salesforce status-page [37] offers already such reporting.

V. TIME BETWEEN FAILURE AND TO RECOVERY

We investigate the difference in cloud service reliability across different datasets and layers of abstraction and make the following observations:

O-3: Present web and cloud services are more reliable than past web services.

O-4: Cloud services are less reliable than web services, indicating that web services, built on top of cloud services, use additional fault-tolerance mechanisms.

O-5: User-reported failures occur more frequently than operator-reported ones but are shorter.

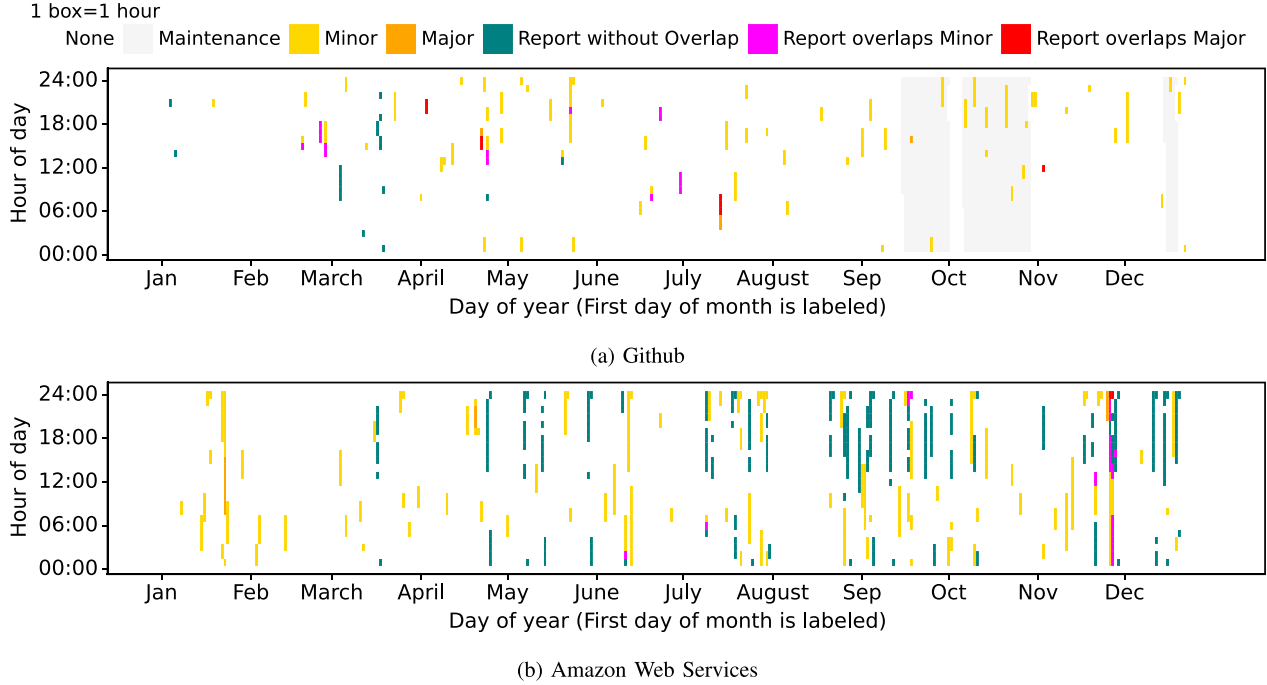


Fig. 5. Failure ranges detected from user- and operator-reported failures. Operators report maintenance, minor, and major failures. Users report incidents, which we process for this work and check for overlaps with operator reports.

O-6: The median recovery time of web and cloud hasn't improved in the last 20 years, but the tail recovery time has improved significantly.

We compare the data from the 5 new datasets we introduce with the service reliability data from the Failure Trace Archive (FTA) [6] and the reliability of datacenter internal workloads from Google [30]. We use the Median Time Between Failure (MTBF), Tail [P95] Time Between Failure (TTBF), Median Failure Duration (MFD) (We use the term Failure Duration instead of Time to Recovery as in MTTR), and Tail [P95] Failure Duration (TFD) metrics to compare the datasets. These metrics capture properties about both service availability and service recovery which contribute to reliability. We choose P95 for the tail as failures are rare events, which means there are at most 100 s of them per service. Using higher values like P99 would only capture the worst couple of events and skew the results.

Comparing the reliability across different levels of abstractions enables us to investigate if services offered at higher abstraction levels (e.g.: Applications) are more reliable than the those offered at lower abstraction levels (e.g.: Infrastructure). Analyzing reliability of both median and tail failures and comparing to the past enables us to investigate where the community has investigated resources to improve reliability.

Fig. 6 depicts boxplots of the MTBF, TTBF, MFD, and TFD of all the traces in each category. The cloud user and operator categories have individual data points instead of a boxplot as those categories only contain 3 traces each (AWS, Azure, and GCP). We

The median MTBF of FTA is 10x lower than the median MTBF of the Web User category and 100x lower than the Web Operator category in Fig. 6(a). The median TTBF of FTA at

3 days is 3x longer than the median TTBF of the Web User Category, but over 10x lower than the median TTBF of the Web Operator category in Fig. 6(b). This indicates that failure in current services occur much more infrequently than failures in the past (O-3).

The MTBF of cloud services is lower than the MTBF of web services, except for user reports for web services (O-4). This implies that web services which want reliability need to use fault tolerance mechanisms of their own to operate in the presence of cloud faults. These mechanisms can include replicas on multiple clouds and failover mechanisms when some cloud services fail.

User-reported failures have a lower MTBF throughout the board for both user- and operator-reported categories (O-5). This indicates that operators do not do report failures which don't affect a significant number of users. It could also indicate that the problem is with the users end of the network or the users device itself.

In Fig. 6(c), the median MFD of current web services is either the same (Web User) or 5x higher (Web Operator). The situation is similar for cloud services (O-6). This indicates that the industry has made little progress in improving recovery time of the median failure in the last 20 years. However, the situation is much better for TFD. The median TFD of both web and cloud services is 4-10x better.

VI. PATTERNS OVER TIME

We investigate failure occurrences at different times during the week and make the following observations:

O-7: Current web and cloud services fail more on weekdays compared to the past when the failure was evenly distributed across all the days.

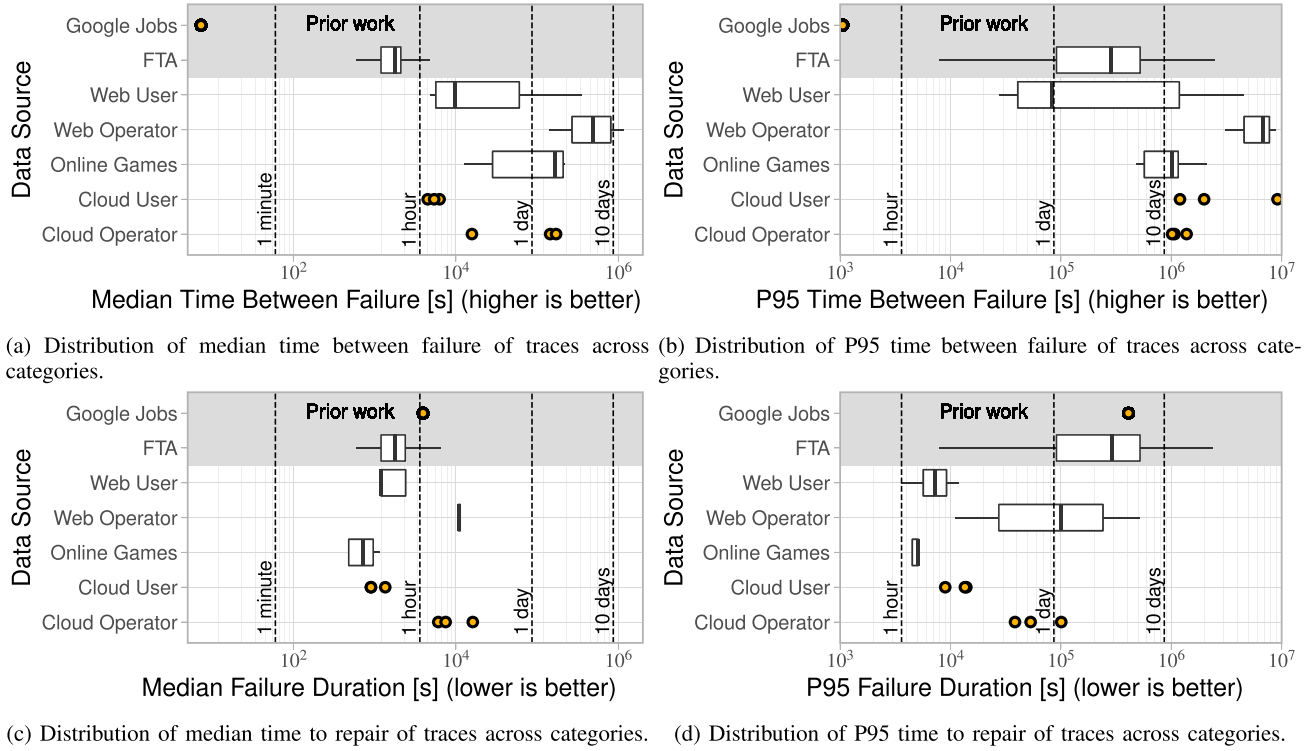


Fig. 6. Statistical properties of the time between failures (time between the end of a failure range and the start of the next one) and failure duration (time between the start of a failure range and its end). Entries taken from prior work are marked with a dark background. Logarithmic horizontal axis, with visual aids for minute, hour, day, and 10 days.

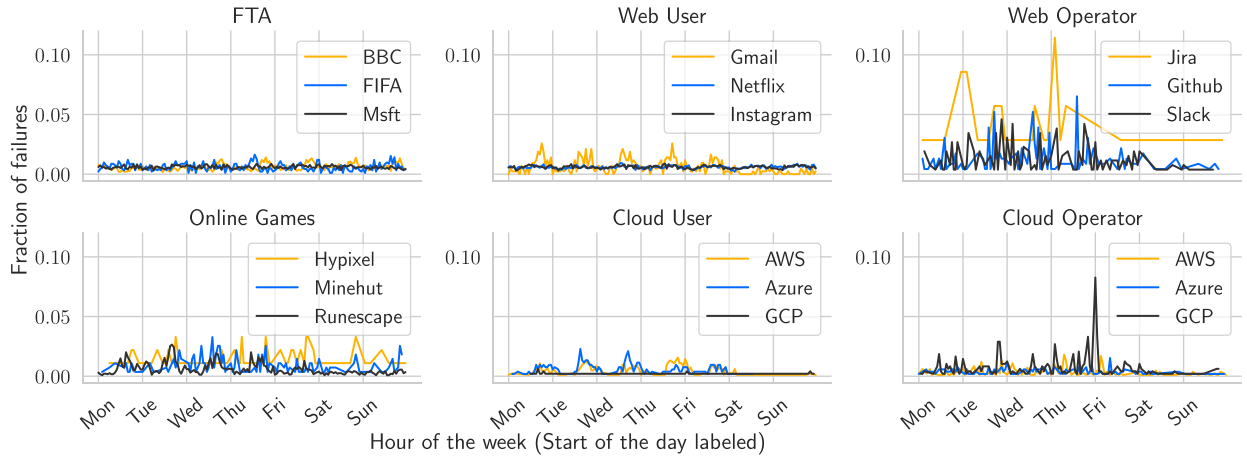


Fig. 7. Weekly trend of failure aggregated per hour. Breaks on the horizontal axis at the start of each day.

O-8: Both web and cloud services fail more in the evening (US East) / afternoon (US West).

O-9: Web services do not display clear modes of early operation and deprecation like hardware for the duration of data we collect.

Comparing the number of failures across the week for current and past services gives us a picture of the operations of these services. We investigate if services fail more when they are used more, e.g., entertainment services in the evening. We also investigate if services fail less on the weekends, which indicates

that the operations processes ensure that new versions are not deployed at the end of the week when engineers do not work.

Fig. 7 depicts the failure pattern of service failure in 6 categories across the week. We only depict 3 representative service in each category. The failure aggregated at one hour intervals based on the time of the week they occur. The horizontal axis of the plot denotes the hour of the week, with the breaks representing the start of every day. The vertical axis represents the fraction of total failures that occurred in a specific hour.

We observe that web and cloud services fail more often during the weekdays than during the weekends. This is a contrast to the

FTA, where failures are uniformly distributed across the week (O-7). This indicates that current services have better operations than in the past, which ensure new versions are not deployed on or close to the weekends. Better operations means better service version deployment processes, better quality assurance, and other such processes. We conjecture that the failures are caused by new service version deployment and not hardware failures or usage pattern changes, because hardware and users do not change on the short week-level time span.

We also observe clear spikes in the middle of the week (Tue-Thu) for Web Operator, Cloud User, and Cloud Operator indicating that new versions are deployed in the middle of the week, giving engineers time to tackle problems when they occur.

We also observe that both web and cloud failures occur in the evenings. Notice that the spikes are very close to the day breaks which indicate midnight. The spike for GCP is particularly visible in the Cloud Operator plot. We believe this is an artifact of our data collection process. We collect data from sensors located on the US East coast. Evening on the US East coast is afternoon on the US West coast. The users of services we investigate are mostly from the USA and this is the time when the largest number of users are online. Large engineering offices of most of the services we investigate are located on the US West coast, which leads us to conjecture that new service versions are deployed when its afternoon in US West.

The online game services hypixel and minehut are more likely to fail at the end of the week, which is different from web and cloud services. This is likely because users play games on the weekends. Runescape does not show this trend and fails more often during weekdays like other services.

Both Schroeder [38] in 2006, and Amvrosiadis [29] in 2019 consider trace duration important. Schroeder to differentiate between early and late life systems. Amvrosiadis to capture the long-term time dynamics of longer than a week. Our multiple-year long data capture duration means we capture patterns of longer than a week making our traces a realistic and less biased input than only week long traces.

We do not observe any distinct long-term operational modes such as early stage (infancy, 6 months to 2 years) and late stage (obsolescence, last 1-2 years). This is because none of the services that we collect data from have become obsolete during our collection period. The services we investigate are much larger than any single hardware cluster and are composed of multiple services themselves. We cannot differentiate these parts well enough to investigate their operational phases well at this time.

VII. FAILURE SEVERITY AND IMPACT ON FAILURE DURATION

We investigate failure severity and its relation to failure duration and make the following observations:

- O-10: Operators report most failures to be low severity, whereas most failures are of medium severity going by user reports and user counts.
- O-11: User reports and online player count denominated failures have their duration positively correlated with the severity. Long operator-reported failures are of low severity.

The severity of failures is representative of how many users and how many components of a service a failure effects. We describe how we interpret and normalize severity across datasets in Section III-D. We interpret failure severity from user reports based on the number of report; from operator reports based on the number of services and the operators self-reported severity; and from online games based on reduction in player counts.

Understanding failure severity helps engineers evaluate the reliability of a service and plan resource allocation. For example, many low-severity failures indicate that quality assurance and development processes must be improved over the long term. Many high-severity failures indicate that reliability should be the development focus, even to the exclusion of other features.

Fig. 9 depicts the ECDF of failures for three categories. The horizontal axis is the normalized severity. We describe how we arrive at the normalized severity score in Section III-D. We observe that web operators report most failures to be not severe. On the other hand, both online games and web users report most failures to be of moderate severity (O-10). This indicates that users' perception of failure is different from the operators perception. For example, an operator might only be seeing a slight drop in user interaction leading them to classify a failure as low severity. However, the failure might be affecting only light users of the service, who, while numerous, might not show up in the metrics as significant because they make few requests each.

Fig. 8 uses a scatter plot to depict the relation between the severity and duration of failures. For both web users and online games categories, failure duration exhibits a mild positive correlation with severity. This indicates that a more severe failure is more likely to last longer. However, the correlation is not strong, with a Pearson correlation of 0.3 (O-11). The web operator category does not exhibit any correlation between failure duration and severity. This is likely because operators report most failures to be of low severity.

VIII. FAILURE SYMPTOMS AND CAUSES

We investigate the root causes of cloud failures and the symptoms as reported by the user and make the following observations:

- O-12: Most users report a low-information failure symptom and access the service from a mobile phone.
- O-13: Most cloud failures are caused by configuration errors, followed by overloads and network issues.
- O-14: Infrastructure services fail more often than user-facing services.

Identifying the root cause of failures allows organizations to better allocate infrastructure and developer resources. For example, hardware failures indicate the need to invest in fault-tolerance techniques such as checkpointing and replication. Code and configuration bugs indicate the need for better developer tools and quality assurance mechanisms. We analyze natural language failure data from 2020 for both user reports and infrastructure-cloud (AWS, Azure, GCP) operator reports of failure.

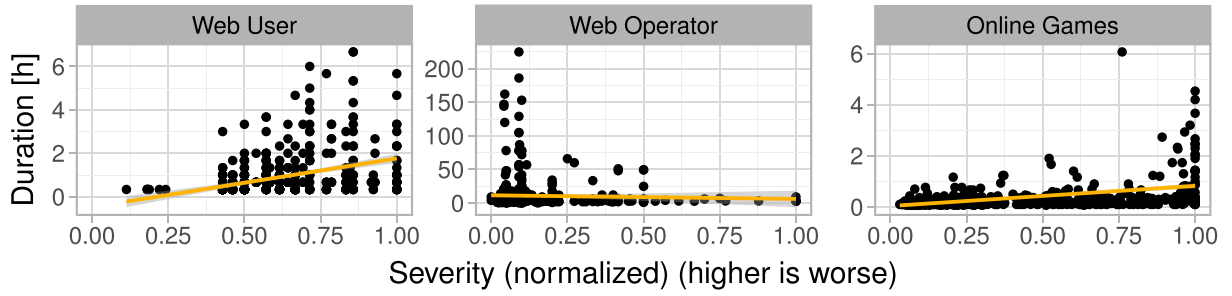


Fig. 8. Failure duration compared to number of players for online games.

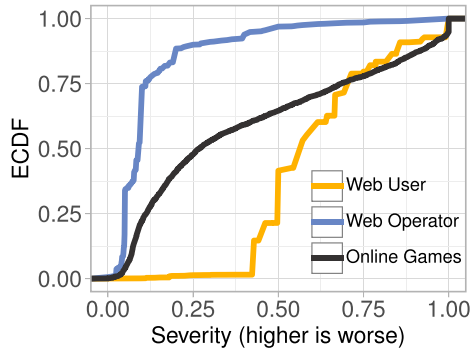


Fig. 9. CDF of failure severity, different services.

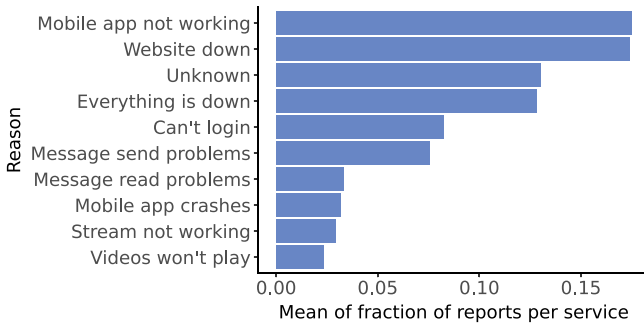


Fig. 10. User-reported symptom of failure.

Outage Report provides users the choice to choose the symptom of failure they experience while reporting. These symptoms are unique to the service such as “Mobile app not working” for Instagram and “Unable to push” for Github.

We summarize the reported symptoms across all services for the top ten symptoms. To compute this summary, we first compute the fraction of reports of each service which were reported due to a particular symptom. Then, for all symptoms of all services, we compute the mean fraction by adding the fraction of the symptom for all services and dividing by twelve, the number of services. We plot the ten symptoms with the highest mean in Fig. 10.

We observe that the most popular failure symptom is “Mobile app not working”. “Mobile app crashes” is also another reason further down the list. The likely reason for this is that for all services except Apple and Github, the mobile app is a major

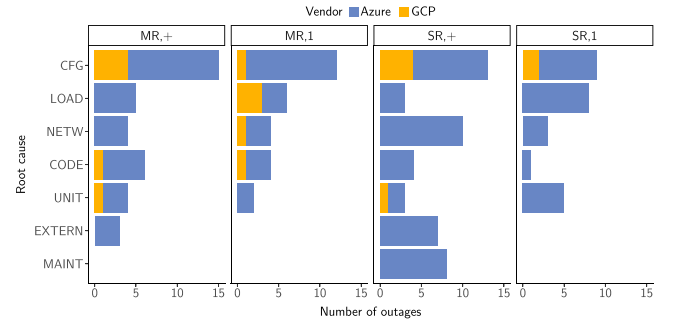


Fig. 11. Root cause of failures, along with number of affected regions, for cloud providers.

way people interact with these services. We do not know if the problem is with the mobile app or the cloud service behind the app. In either case, this is a kind of failure that should be further studied by the community. We should investigate if mobile applications present opportunities to deal with failure that were previously unavailable. The second most popular symptom is “Website down” (O-12). This likely refers to the website not loading at all or being very slow to load that it is unusable. We expected this to be a popular symptom as it is a catchall for all kinds of failures that might crop up in application accessed via the browser. Another such catch all which is further down the list is “Everything is down.” All reports without a reason were categorized as “Unknown.”

Fig. 11 depicts the root cause given by two cloud operators (Azure, GCP) in their own failure reports. We perform this analysis manually over a one year subset of the data. We analyze the distribution of reasons across root causes and impact levels. We identify seven classes of root causes: UNIT (individual nodes, instances, or clusters, not necessarily hardware), NETW (related to the internal or external network), MAINT (side effects caused by maintenance), LOAD (increased load on the service), EXTERN (external causes, i.e. environmental or third-party), CODE (code errors/bugs), and CFG (configuration errors). We further separate the outages by vendor, indicated by the color of the bar in Fig. 3. We do not include AWS, as we do not have sufficient data regarding root causes from AWS (a cause was only specified for 2.1% of AWS events). Excluded from the plot are outages that did not provide a root cause (a total of 122 events, 45.69%), a range (5 events, 1.87%), or the number of affected services (4 events, 1.5%).

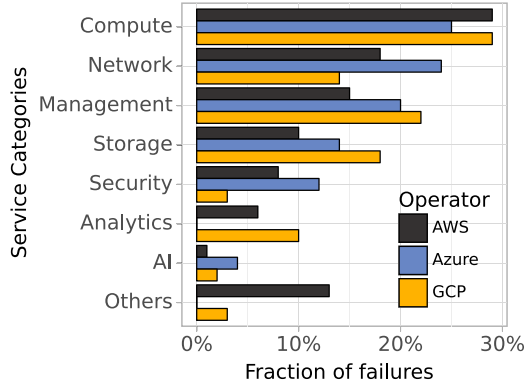


Fig. 12. Categories of services affected by failures.

The majority of outages across all levels of impact are caused by configuration errors. For multi-regional outages, the configuration category accounts for the majority by a wide margin. (O-13) On the other hand, for single-region outages, there are multiple leading causes: apart from configuration errors, outages affecting one service are also caused by increased load and failing instances, and those affecting more than one service are also caused by network errors and maintenance side effects.

We observe that load-related outages tend to happen starting between midday on Wednesday and Thursday evening, with smaller peaks during the night on other days. Most outages caused by misconfiguration happen in the first half of the week, from mid-Monday until midnight on Thursday. This could perhaps be because large code changes are usually introduced during the first few days of the week. The major peaks for these outages generally occur around midnight. The reason for this could be that deployments happen during the night, when it is likely that fewer customers will be using the service; as Langford et al. found, there is a significant difference in traffic during the day and during the night [39]. It could also be that changes are deployed during the day, but there is a delay before bugs appear.

We identify the service categories that were affected by the failures. We use LLM-enabled analysis of operator failure reports (AWS, Azure, GCP) using the method we describe in Section III-C. The results of our analysis are depicted in Fig. 12. We classify cloud services into 8 categories. We then use LLMs to classify the failure reports into these categories. We observe that infrastructure services like compute services (e.g., EC2 in AWS), network services (e.g., VPC in AWS), and the management plane fail most often. Services which offer a higher level of abstraction and are more user-facing, such as Analytics and AI fail the least (O-14). This supports our earlier observations in Section V that user-facing Web services have higher inter-failure duration than infrastructure cloud services. The confusion matrix for service category extraction using claude-3.5-haiku is depicted in Fig. 13. We observe more than 80% true positives for 6 out of the 8 categories. Most of the false negatives for these 6 categories are due to the failed services being classified as Other. For the other 2 categories, Management and Other, the true positives are only 14.7% and 66.7%, respectively. Management service failures are often misclassified as Compute or

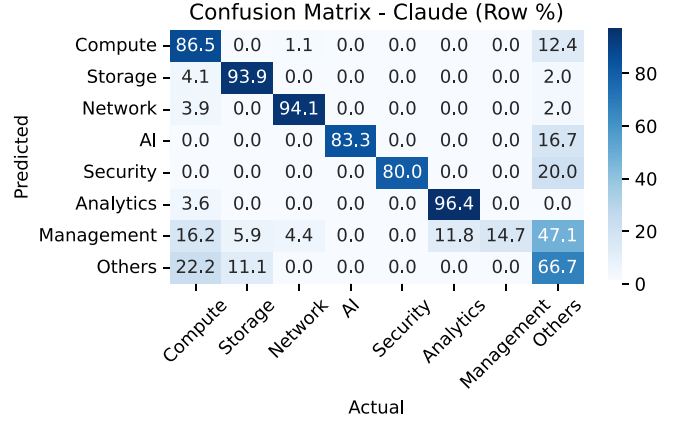


Fig. 13. Confusion matrix for service category, using Claude.

Other service failures. Other service failures are misclassified as Compute or Storage failures. We conjecture that this is due to the low number of training samples compared to the diversity of issues available for these categories.

IX. USING FAILURE TRACES TO EVALUATE THE IMPACT OF FAILURES AND CHECKPOINTING ON BATCH JOBS

We demonstrate how the traces we release in this work can be used to investigate the fault-tolerance properties of a system or an operational technique. This addresses the challenge highlighted in the introduction, in which fault-tolerance papers use synthetic or nonpublic traces, making comparison difficult.

We investigate the efficacy of checkpointing industrial batch jobs at tolerating the failures of external services used by those jobs. Industrial batch jobs often access external services such as storage services like Amazon S3 [41], data catalogs like Apache Iceberg [42], customer data repositories like Salesforce [43], and ERP systems like SAP [44]. We make the following observations:

- O-15: Using checkpointing can decrease the runtime of a batch workload by up to 69% and the number of failed tasks by 72% in the presence of external service failures.
- O-16: The impact of failures and checkpointing is not consistent across all metrics. Using checkpointing can significantly reduce the number of terminated tasks (52%) while reducing the total runtime by only 15%.

A. Checkpointing

Checkpointing is a well-known technique that enables tasks to progress despite failures [45]. When using checkpointing, snapshots of a task are taken periodically. In the event of failure, a task can be restarted from its last snapshot rather than from the start, saving a significant amount of time. However, checkpointing is not without costs. Creating a snapshot can take significant time, depending on the task. Because of this, creating snapshots too frequently can result in the overhead of checkpoints exceeding its gains. In this section, we compare the impact of different checkpointing models on the performance of different data centers exposed to different failure models.

TABLE II

FAILURE TRACES USED IN EXPERIMENTS. TBF: AVERAGE TIME BETWEEN FAILURES, DURATION: THE AVERAGE DURATION OF A FAILURE, INTENSITY: THE AVERAGE RATIO OF HOSTS THAT ARE EFFECTED BY AN FAILURE.

Failure trace	TBF	Duration	Intensity
Facebook Messenger	2 days 15:50:22	00:36:51	0.299
Gmail	1 days 06:57:13	00:26:48	0.572
Whatsapp	0 days 14:35:48	00:26:44	0.579

B. Experiment Setup

Experiments are conducted using the OpenDC simulator,¹ an open-source data center simulator. Using OpenDC, we are able to replay the same workload trace in combination with different failure traces and checkpointing models.

1) *Workload*: To understand the impact of failures on data centers, we use a large-scale industrial batch workload based on the Google Borg infrastructure [40]. We use a subset of the dataset containing all tasks executed on cell-a located in New York.² The workload contains 14,867,803 tasks executed on 1534 machines. On average, tasks run for two hours.

2) *Failure Traces*: Three different failure traces are used with a low (Whatsapp), medium (Gmail), and high (FaceBook Messenger) frequency of failures (see Table II). The starting point of the failure trace determines when which failures occur and can thus influence the results. To mitigate this problem, each experiment is run 5 times, starting at different points in the failure trace.

3) *Checkpointing Models*: We compare the impact of eight different checkpoint models that differ in the frequency of creating snapshots. The “no checkpointing” model does not make any checkpoints and is used as a baseline. The next six models use a static interval of 5, 10, 30 minutes, and 1, 5, 10 hours between snapshots. Finally, the “adaptive” model uses a dynamic interval between snapshots, starting at 5 minute and increasing the interval by multiplying it by 1.5 after each snapshot. After a failure, the interval is restored to 5 minutes.

C. Metrics

The impact of checkpointing is determined using five metrics that focus on performance, quality of service, and sustainability.

1) *Runtime*: The time required to complete all tasks in a workload. This is also often referred to as the makespan.

2) *Task Delay*: The average delay of a task. This is calculated by subtracting the theoretical best execution time (the task is executed directly on submission and is never delayed) from the task’s actual execution time. This metric provides insight into the customer’s quality of service.

3) *Tasks Terminated*: The number of tasks that the data center terminated due to excessive failures. In our experiments, tasks are terminated after failing more than 10 times.

4) *Energy Usage*: The total energy used by the data center during the workload. Note: we assume that when a machine fails, it will keep using energy as if it were idle.

5) *Carbon Emission*: The total carbon emitted during the workload caused by energy usage. Carbon emission is calculated using traces collected from ENTSO-E.³ We use carbon intensity traces from the Dutch power grid for 2022.

D. Experiment Results

Fig. 14 depicts the impact of using different checkpointing models on the execution of the Borg workload when subjected to different failure traces.

The Impact Of Checkpointing: The best checkpointing model decreases total runtime by 69%, 37%, and 15% when the workload is subjected to low-, medium-, and high-frequency failure traces, respectively. For medium- and high-frequency failure traces, the number of failed tasks decreases by 72% (from 72,000 to 20,000) and 59% (from 136,000 to 56,000), respectively (O-15). We observe similar reductions in other metrics, such as energy usage and carbon emissions. It is important to note that not all checkpointing models perform equally. If the interval is too small, creating snapshots incurs excessive overhead, whereas models with larger intervals do not capture enough snapshots to make a significant difference when failures occur. The best two models are the 30-minute and 1-hour interval models. In terms of average task delay, the 5-minute interval model performs worst, increasing it by up to 50% compared to the best model. In terms of task termination, the 10-hour interval model performs worst, increasing the number of terminated tasks by up to 50% relative to the best model.

Comparison Between Metrics: The impact of checkpointing can vary significantly across metrics. For example, using the 5-minute interval model yields a 52% reduction in the number of tasks terminated when exposed to the high-frequency failure model. However, the total runtime is reduced by only 15% (O-16). This shows that improvements to the full system do not always benefit individual users.

X. USING FAILURE TRACES TO ESTIMATE THE NUMBER OF RETRIES IN SERVICE-BASED APPLICATIONS

We demonstrate how failure traces can be used in microservice-based computer systems experiments and how they impact the results. Most microservices in the datacenter access other microservices from the same application, rather than an external service. Using user-reported failure traces, as we do in this work, is not representative of the behavior of such microservices. *Our results are valid for microservices that access external services also used by end-users.* Many business applications interact with what are normally end-user applications, such as Facebook ads bidders and WhatsApp bots for customer service. Ideally, failures should be injected using traces from various sources, including internal reports, operator reports, and user reports, depending on the type of microservice being evaluated.

Evaluating a chain of microservice, all being subjected to just user reports, our main finding is:

¹opendc.org

²https://console.cloud.google.com/storage/browser/clusterdata_2019_a

³<https://www.entsoe.eu/>

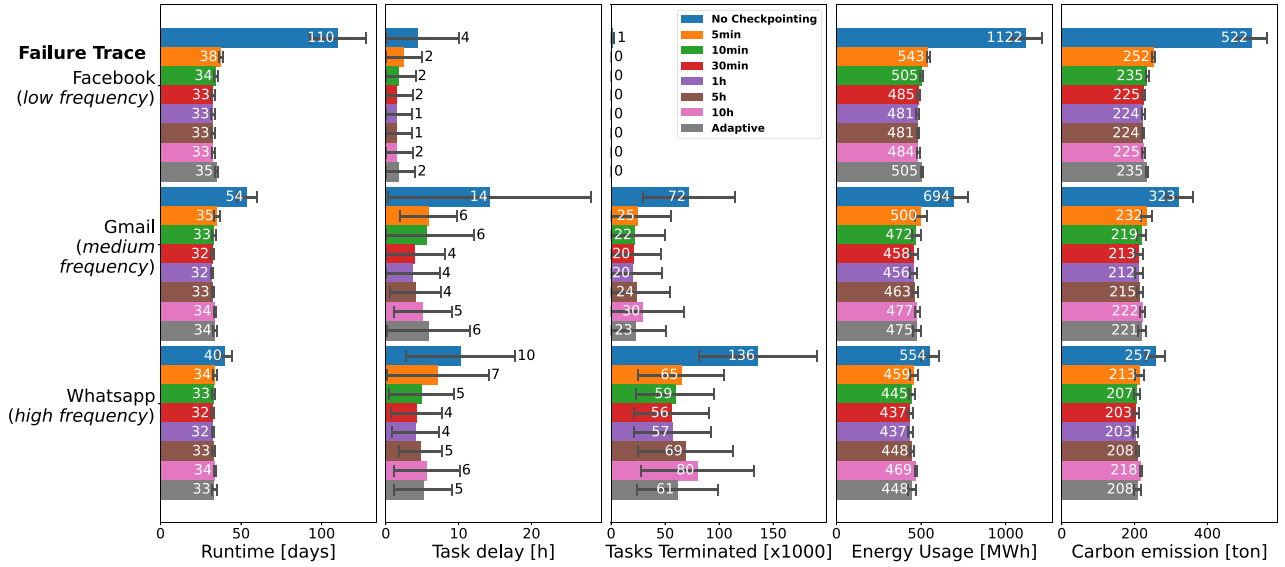


Fig. 14. Impact of checkpoint model and failure trace on the Google Borg workload trace [40] of industrial batch jobs.

O-17: A real trace leads to more than $10\times$ less retries, but $100\times$ more failures compared to a constant failure probability for applications composed of a long chain of services.

Complex cloud applications such as e-commerce, social media and banking are important to daily life. Latency is an important metric for such applications. For example, increased latency reduces the click-through rate of web search [46]. Complex cloud applications are composed of tens, or even hundreds of (micro-)services, working together to perform a task. The latency of the application depends on the latency of each of service. It also depends on how often the services fail. For example, additional latency appears when a service failure leads to a request being retried. Existing work [13], [47] considers *constant failure probability* or similar simple models to estimate the additional latency. We show that using user-reported failure traces produces different results. Specifically, most requests succeed with fewer retries than if a constant failure probability was assumed, but during failure periods, a lot more requests fails.

We conduct experiments with multi-service apps, using trace-based simulation. We interpreting the number of user reports of a service as the probability that a request to the service fails. We scale the number of reports by the number of users of the service at that time. As we do not know the actual number of users, we assume that the number of users follows a diurnal pattern with peak usage in the evening; we approximate this with a shifted sine wave. After scaling the trace, we normalize it into probabilities $[0, 1]$ by dividing each data point by the maximum number of user reports in a single time period (20 minutes in our case). We thus obtain a failure probability associated with every 20-minute period in the trace.

We choose three app structures—for each, a number of microservices and their interdependencies. Many microservice applications are composed of simple structures [48] such as (1) monolith (a single microservice called by a client), (2) fanout (a client makes multiple requests to a microservice all of which

need to succeed), and (3) long chain (a client calls a microservice, which calls another, and so in a long chain). We simulate failures in each of these structures and measure the impact on latency.

We use the metric number of retries as an indicator of the tail latency experienced by the service. Tail-latency is particularly important in cloud operations [49], where services are possibly used by millions of users daily. We assume each microservice and client comes with a retry oracle which knows when to retry. Hedging [13] and failure detection algorithms [50] have been studied extensively, and our results complement that work. We also assume that cost of retry dominates the latency, compared to the actual request-processing time; this holds for short-running apps [51].

We evaluate the number of retries experienced by a long chain workload structure comprised of 5 microservices. The evaluation for other structures is in the technical report. At each stage, we allow a maximum of 3 retries, which is common in practise. Reaching the maximum number of retries at a stage means that the previous stage has to retry again. When the retries of the first stage are exhausted, the request is considered to have *failed*. Fig. 15 depicts the results. The horizontal axis represent the number of retries with a special marker for failed requests. The vertical axis is logarithmic axis depicting the fraction of requests. A point on the plot represents the fraction of requests that required a certain amount of retries to succeed. We run the experiment with two different workloads: (new) the Instagram use-reported failure trace and (traditional) a constant failure trace. The constant failure probability we chose is 1% ($P(f) = 0.01$). The arrival pattern is diurnal, peaking in the evening; we approximate this with a shifted sine wave.

We find that the number of requests that succeed is, in total across all possible retry-counts and also individually for retry-counts 1 and 2, more than $10\times$ lower for the Instagram trace compared to the constant failure probability (O-17). Requests which require 4 retries are non-existent for the Instagram trace, but they appear prominently with constant failure probability.

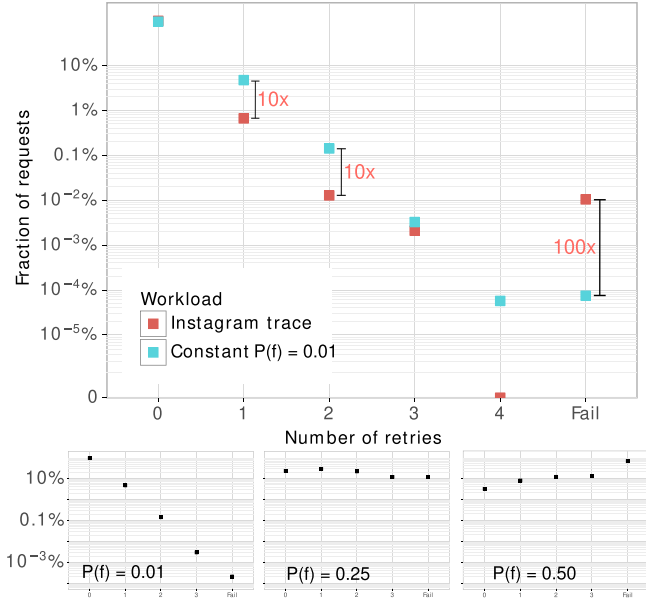


Fig. 15. Distribution of retries for a long chain chain microservice structure with 5 microservices. At the bottom are the distributions at different failure probabilities indicating why the distribution for the trace looks like it does. In particular, notice the large number of failed requests at high failure probability and for the trace. The vertical axis is logarithmic.

Significant differences appear also in failed requests; $100\times$ more requests fail with the Instagram trace compared to the constant failure probability. The number of requests which succeed with 3 retries is very close for both the Instagram trace and constant failure probability. To understand the unique shape of the failure distribution exhibited when using the Instagram trace, we plot the retry distribution with different failure probabilities found in the trace at the bottom of Fig. 15. Notice the flat line at 25% failure probability, which is the likely cause of the high number of requests that succeed with 3 retries for the Instagram trace. Similarly, at 50% failure probability, the number of failed requests becomes very high. Such periods with high failure probability lead the high number of failed requests for the Instagram trace.

XI. DISCUSSION AND THREATS TO VALIDITY

Data Accuracy: We collect data from different sources such as operator status pages and user-report crowdsourcing websites. Each of these sources has its interpretation of failure, making it challenging to evaluate the accuracy of data with respect to the ground truth. We compare different failure report sources for the same service in Section IV as a sanity check to ensure that our data is accurate at least when it comes to major failures.

Crowdsourced Data Collection: Lack of data from multiple crowdsourced data sources biases our work to the pool of users who report to the particular crowdsourcing services we collected the data from, Outage Report and DownRightNow. It also exposes the number of user reports to vagaries such as changing search-engine ranking (i.e., when Outage Report is not the first result), changing social media popularity of the service, and

other causes we cannot control. For example, the reduction in number of user reports could be due to the reduced popularity of the service and not to fewer actual failures.

Comparison to Old Datasets: We compare our newly collected data to a previous dataset from the Failure Trace Archive (FTA) [6]. The FTA dataset was collected by visiting different websites periodically from a central server. That is a different method from using crowdsourced and operator reports. This difference makes high-precision numerical comparisons between the datasets error-prone. However, we believe our findings hold because the failure during and time between failures of the two datasets are at least an order of magnitude different.

Comparison Between Different Services: We know that comparing multiple services and cloud operators is a non-trivial pursuit. Our selective experiments and findings should not be interpreted as stating conclusively that one service or operator is better than the others. We do not have sufficient data to investigate such claims. Such data could arise from a community-wide study on multi-year data and would be facilitated by the release of ground-truth data by the cloud operators. **Appropriate Use of Failure Traces for Evaluation:** The failure traces released in this work capture cloud service failures as perceived by an external client, rather than internal infrastructure faults. They are primarily suited for comparative evaluation of client-side and service-level fault-tolerance mechanisms, including retry and backoff policies, circuit breakers, and resilience to cascading failures.

Using these traces for end-to-end evaluation of a complete computing system requires composition with additional failure sources that are not represented here, such as node/hardware faults and application-level software defects. In Section IX, we therefore intentionally restrict the scope to external service failures to demonstrate how such traces can be integrated into an evaluation pipeline; conclusions from this experiment should be interpreted as conditional on that scope and are not intended to represent full-system reliability.

These traces should not be used to draw conclusions about environments where external service dependencies are not a dominant failure driver (e.g., HPC workloads), or to infer node-level reliability without complementary data.

XII. RELATED WORK

Understanding failures is a long-lasting area of interest in computer systems research. We conduct a systematic literature survey [67] of failure studies related to distributed systems for the articles published in top conferences and journals between 2010 and 2024. Table III summarizes the 21 articles we found closest to our failure characterization; although only 3 focus on clouds, the others characterize failures in large-scale systems. The mapping of our study to the table emphasizes the novel aspects of our work: unique data and an unique mix of approaches for analysis, for a cloud workload; this leads to new findings.

Closest to our is the Failure Trace Archive [6] which collected data about the availability of networks of workstations, websites, and some HPC clusters till 2009. Our data about web and cloud service is qualitatively and quantitatively different. We collect

TABLE III
OVERVIEW OF RELATED WORK. (TW = THIS WORK)

Feature	Type	Articles	Comments
Data Source	Crowdsourced	TW-only	Users' experience with cloud services.
	Cloud Providers	[52] & TW	Data available from cloud provider or the news.
	Multiple Clusters	[4, 6, 53]–[56]	Data for multi-cluster infrastructure.
	Single Cluster	[3, 19, 57]–[62]	Data for infrastructure of a single cluster.
	User Reports	[5, 52, 63]–[66]	Bug reports for individual software.
Workload	Cloud	[3, 52, 66] & TW	Data from cloud datacenters or cloud applications.
	HPC	[4, 6, 19, 53, 55, 56, 59]–[62]	Supercomputer applications and hardware.
	Big Data	[4, 5, 58, 63]	Data processing systems like MapReduce.
	Enterprise	[6, 54, 57, 64, 65]	From enterprise vendors such as IBM.
Analysis	Distribution	[3]–[6, 19, 52]–[60, 62] & TW	Report empirical distributions of failure features.
	Trend	[19, 52, 53, 55]–[61, 65, 66] & TW	Report trends over time, location, lifetime, etc.
	Categorization	[3]–[5, 52]–[58, 61]–[66] & TW	Categorize failures based on severity, root cause, etc.

and analyze data at the service level, a higher level abstraction than hardware. The world has also changed significantly, with online services involved in every part of life. Networks of workstations and HPC clusters are not involved in daily life to the same extent. The quantitative differences are analyzed in Sections V and VI.

Second closest to our work, Gunawi et al. [52] analyze cloud failures over a multi-year period, using news reports as input data. Complementary to our time-related analysis, they find that the distribution of failure duration in a bad year skews higher than in an average year. Most of their work focuses on root-cause analysis, which is outside the scope of this work. Their analysis captures failures with the highest impact, but misses other failures [68] that, although not large enough to make the news cycle, are important to users, engineers, and thus also researchers.

Failure Characterization Studies: User-reported bugs for individual software have been widely studied. Examples include operational bugs study from Microsoft Teams [69] and the OpenStack fault injection study [70]. Bug reports for open-source software have also been analyzed [5]. These works do not study the long-term availability of multiple cloud services, which we do. Although tools for collecting and analyzing the client-side failure behavior of cloud services exist [71], how users experience the behavior of cloud services is still virtually unknown. This work aims to address this knowledge gap.

We compare our work to other studies on multi-cluster environments. Several studies observe the presence of short and/or long interarrival times [54], bursty and correlated failures [6], time-patterns over days of the week and months of the year [53], etc. But our study complements this body of work: the quantitative results we present are often different, e.g., minute-length failures for most SaaS services, in contrast to the hours to days observed for multi-clusters.

Hardware Failure Characterization: The reliability of CPUs, GPUs [55], [72], [73], [74], [75], [76], and SSDs [1], [77], [78], [79] has been extensively characterized. The MTBF of GPUs (15 - 160 hours) is higher than for web services (< 24 hours) [55], [74]. Even bit errors have MTBF of over 48 hours [73]. Software drivers often cause hardware failures [55], [74]; we conjecture that the higher incidence of software errors is due to the broad

scope of use cases and the complexity of the software itself. The error rates of SSDs are extremely high compared to web services [1]. Whole machines fail much less often. They operate reliably, with a 3-year median MTBF [75].

Dependability benchmarking: Fault injection-based benchmarking has seen some success in the evaluation of single-machine software [80]. For distributed systems, fault injection methods such as MATCH [81] for MPI, Jepsen for databases [82], and Monarch [83] for distributed file systems have been proposed. Our work complements these methods by enabling their use in whole-application evaluation, augmenting them with information on when to inject failures and at what intensity, which can be derived from traces. This is similar to how workload traces are used in performance evaluation, in addition to benchmarks [84].

Trace Archives and Datasets: Multiple trace archives have been created to benefit the community. Example of trace archives include parallel workloads archive [24], grid workloads archive [25], cloud software bugs [5], workflow trace archive [26], and SNIA IOTTA [85]. Multiple datasets have also been published independently to benefit the community such as the Supercloud dataset [86], Twitter KV-traces [87], CINECA Marconi dataset [88], and HPC-ML workloads [89]. Cloud providers like Google [90], Microsoft [91], and Alibaba [92] have also released cluster usage traces. All these traces have been used in thousands of papers and have pushed the field forward. Several studies have systematically investigated performance variability in the cloud [93], [94], [95]. However, they do not investigate failures.

XIII. CONCLUSION

In digital societies, cloud services are pervasive in every industry and affect most people. However, we do not understand their reliability well, not as well as we do for, e.g., cars. We propose collecting data about the availability of popular online services as the first step towards solving this problem.

We collect long-term data about failures of online services such as infrastructure clouds (AWS, Azure, GCP), online services (Github, Atlassian, Facebook, Netflix, etc.), and online games (Runescape, Minecraft). We collect this data from two

vantage points, the service operators and the users. We summarize data and their sources in Table I.

We demonstrate the structure and basic utility of this data in Section II. We describe our data collection, cleaning, interpretation, and analysis process in Section III. We also describe our trace format. We validate the data by comparing the data for the same failures from operator reports and user reports for two services, Github and AWS, in Section IV.

We analyze this data through a standard MTBF and MTTR reliability analysis in Section V, time patterns in Section VI, failure severity in Section VII, user-reported symptoms, and operator reported symptoms in Section VIII. This analysis results in 14 observations.

We demonstrate the utility of the failure traces we collect by evaluating the impact of traces on checkpointing policies in Section IX and retry policies in Section X. This simulator-based experimental analysis leads to 3 observations.

Finally, we release the failure data, analysis code, and simulation code as open-source artifacts at <https://github.com/atlarge-research/cloud-uptime-archive>. The dataset is also available on Zenodo at <https://doi.org/10.5281/zenodo.14712441>. We call the community to enhance this dataset with new failure data sources, analysis techniques, and using the traces in the evaluation of future fault-tolerant systems.

REFERENCES

- [1] S. Maneas, K. Mahdavi, T. Emami, and B. Schroeder, "Reliability of SSDs in enterprise storage systems: A large-scale field study," *ACM Trans. Storage*, vol. 17, no. 1, pp. 3:1–3:27, 2021.
- [2] P. H. Hochschild et al., "Cores that don't count," in *Proc. HotOS '21: Workshop Hot Topics Operating Syst.*, 2021, pp. 9–16.
- [3] A. Rosà, L. Y. Chen, and W. Binder, "Understanding the dark side of Big Data clusters: An analysis beyond failures," in *Proc. 45th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2015, pp. 207–218.
- [4] N. El-Sayed, H. Zhu, and B. Schroeder, "Learning from failure across multiple clusters: A trace-driven approach to understanding, predicting, and mitigating job terminations," in *Proc. 37th IEEE Int. Conf. Distrib. Comput. Syst.*, 2017, pp. 1333–1344.
- [5] H. S. Gunawi et al., "What bugs live in the cloud? A study of 3000 issues in cloud systems," in *Proc. ACM Symp. Cloud Comput.*, 2014, pp. 7:1–7:14.
- [6] B. Javadi, D. Kondo, A. Iosup, and D. H. J. Epema, "The failure trace archive: Enabling the comparison of failure measurements and models of distributed systems," *J. Parallel Distrib. Comput.*, vol. 73, no. 8, pp. 1208–1223, 2013.
- [7] "https://www.nhtsa.gov/," 2024. [Online]. Available: <https://www.nhtsa.gov/>
- [8] "National transportation safety board," 2024. [Online]. Available: <https://www.nts.gov/Pages/home.aspx>
- [9] S. J. Vaughan-Nichols, "Why atlassian failed so hard," 2022. [Online]. Available: <https://thenewstack.io/why-atlassian-failed-so-hard/>
- [10] "Zoom just failed its biggest test," 2020. [Online]. Available: <https://www.inc.com/jason-aten/zoom-just-failed-its-biggest-test-its-apology-is-an-extraordinary-example-of-emotional-intelligence.html>
- [11] R. G. International, "Scheduling software sidelines ns services," 2022. [Online]. Available: <https://www.railwaygazette.com/passenger/scheduling-software-failure-sidelines-ns-services/61304.article>
- [12] T. Sterling, "Dutch investigate kpn network outage, company blames software bug," 2019. [Online]. Available: <https://www.reuters.com/article/world/dutch-investigate-kpn-network-outage-company-blames-software-bug-idUSKCN1TQ1FK/>
- [13] M. Primorac, K. J. Argyraki, and E. Bugnion, "When to hedge in interactive services," in *Proc. 18th USENIX Symp. Networked Syst. Des. Implementation*, 2021, pp. 373–387.
- [14] J. Mohan, A. Phanishayee, and V. Chidambaram, "CheckFreq: Frequent, fine-grained DNN checkpointing," in *Proc. 19th USENIX Conf. File Storage Technol.*, 2021, pp. 203–216.
- [15] A. Maurya, R. Underwood, M. M. Rafique, F. Cappello, and B. Nicolae, "DataStates-LLM: Lazy asynchronous checkpointing for large language models," in *Proc. 33rd Int. Symp. High-Perform. Parallel Distrib. Comput.*, 2024, pp. 227–239.
- [16] G. Andreadis, F. Mastenbroek, V. van Beek, and A. Iosup, "Capelin: Data-driven compute capacity procurement for cloud datacenters using portfolios of scenarios," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 1, pp. 26–39, Jan. 2022.
- [17] J. Kadupitiya, V. Jadhao, and P. Sharma, "SciSpot: Scientific computing on temporally constrained cloud preemptible VMs," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 3575–3588, Dec. 2022.
- [18] M. Brooker, T. Chen, and F. Ping, "Millions of tiny databases," in *Proc. 17th USENIX Symp. Networked Syst. Des. Implementation*, 2020, pp. 463–478.
- [19] S. Gupta, D. Tiwari, C. Jantzi, J. H. Rogers, and D. Maxwell, "Understanding and exploiting spatial properties of system failures on extreme-scale HPC systems," in *Proc. 45th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2015, pp. 37–44.
- [20] Statista, "Number of monetizable daily active x (formerly twitter) users (mdau) worldwide from 1st quarter 2017 to 2nd quarter 2022," 2024. [Online]. Available: <https://www.statista.com/statistics/970920/monetizable-daily-active-twitter-users-worldwide/>
- [21] B. Dean, "X (twitter) statistics: How many people use x?," 2024. [Online]. Available: <https://backlinko.com/twitter-users>
- [22] "Sensor tower," 2024. [Online]. Available: <https://sensortower.com/>
- [23] D. Ingram, "Fewer people are using elon musk's x as the platform struggles to attract and keep users, according to analysts," 2024. [Online]. Available: <https://www.nbcnews.com/tech/news/fewer-people-using-elon-musk-x-struggles-keep-users-rcna144115>
- [24] D. G. Feitelson, D. Tsafir, and D. Krakov, "Experience with using the parallel workloads archive," *J. Parallel Distrib. Comput.*, vol. 74, no. 10, pp. 2967–2982, 2014.
- [25] A. Iosup et al., "The grid workloads archive," *Future Gener. Comput. Syst.*, vol. 24, no. 7, pp. 672–686, 2008.
- [26] L. Versluis et al., "The workflow trace archive: Open-access data from public and private computing infrastructures," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 9, pp. 2170–2184, Sep. 2020.
- [27] R. Potharaju and N. Jain, "When the network crumbles: An empirical study of cloud network failures and their impact on services," in *Proc. ACM Symp. Cloud Comput.*, 2013, pp. 15:1–15:17.
- [28] Y. Chen et al., "Automatic root cause analysis via large language models for cloud incidents," in *Proc. 19th Eur. Conf. Comput. Syst.*, 2024, pp. 674–688.
- [29] G. Amvrosiadis, J. W. Park, G. R. Ganger, G. A. Gibson, E. Baseman, and N. DeBardeleben, "On the diversity of cluster workloads and its impact on research results," in *Proc. USENIX Annu. Tech. Conf.*, 2018, pp. 533–546.
- [30] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale cluster management at google with borg," in *Proc. 10th Eur. Conf. Comput. Syst.*, 2015, pp. 18:1–18:17.
- [31] P. Huang et al., "Gray failure: The achilles' heel of cloud-scale systems," in *Proc. 16th Workshop Hot Topics Operating Syst.*, 2017, pp. 150–155.
- [32] Github, "Incident on 2020-07-13 04:06 utc," 2020. [Online]. Available: <https://www.githubstatus.com/incidents/j597fw8kv04c>
- [33] T. Anderson, "Github is just like all of us: The week has just started but it needed 4 whole hours of downtime," 2020. [Online]. Available: https://www.theregister.com/2020/07/13/github_takes_some_downtime_availability/
- [34] E. Dwoskin, F. Siddiqui, G. D. Vynck, and J. B. Merrill, "Documents detail plans to gut twitter's workforce," 2022. [Online]. Available: <https://www.washingtonpost.com/technology/2022/10/20/musk-twitter-acquisition-staff-cuts/>
- [35] R. B. Cleveland, W. S. Cleveland, J. E. McRae, and I. Terpenning, "STL: A seasonal-trend decomposition," *J. Off. Stat.*, vol. 6, no. 1, pp. 3–73, 1990.
- [36] E. Bouwers et al., "Getting what you measure," *Commun. ACM*, vol. 55, no. 7, pp. 54–59, 2012.
- [37] Salesforce, "Salesforce services status," 2025. [Online]. Available: https://status.salesforce.com/products/Salesforce_Services
- [38] B. Schroeder and G. A. Gibson, "A large-scale study of failures in high-performance computing systems," *IEEE Trans. Dependable Secur. Comput.*, vol. 7, no. 4, pp. 337–351, Oct.–Dec. 2010.
- [39] J. Langford, L. Li, R. P. McAfee, and K. Papineni, "Cloud control: Voluntary admission control for intranet traffic management," *Inf. Syst. E Bus. Manag.*, vol. 10, no. 3, pp. 295–308, 2012.
- [40] M. Tirmazi et al., "Borg: The next generation," in *Proc. 15th Eur. Conf. Comput. Syst.*, 2020, pp. 1–14.

- [41] M. Vuppapalapati, J. Miron, R. Agarwal, D. Truong, A. Motivala, and T. Cruanes, "Building an elastic query engine on disaggregated storage," in *Proc. 17th USENIX Symp. Netw. Syst. Des. Implementation*, R. Bhagwan and G. Porter, Eds., USENIX Association, 2020, pp. 449–462. [Online]. Available: <https://www.usenix.org/conference/nsdi20/presentation/vuppapalapati>
- [42] A. I. Contributors, "Apache iceberg: The open table format for analytic datasets." 2025. [Online]. Available: <https://iceberg.apache.org/>
- [43] P. C. Das et al., "CDI-E: An elastic cloud service for data engineering," *Proc. VLDB Endow*, vol. 15, no. 12, pp. 3319–3331, 2022. [Online]. Available: <https://www.vldb.org/pvldb/vol15/p3319-moskovich.pdf>
- [44] J. Ivain, "Navigating the SAP data ocean: Demystifying SAP data extraction for databricks integration using ODP," 2024. [Online]. Available: <https://community.databricks.com/t5/technical-blog/navigating-the-sap-data-ocean-demystifying-sap-data-extraction/ba-p/94617>
- [45] R. Koo and S. Toueg, "Checkpointing and rollback-recovery for distributed systems," *IEEE Trans. Softw. Eng.*, vol. SE-13, no. 1, pp. 23–31, Jan. 1987.
- [46] I. Arapakis et al., "Impact of response latency on user behavior in web search," in *Proc. 37th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2014, pp. 103–112.
- [47] A. Vulimiri et al., "Low latency via redundancy," in *Proc. 9th ACM Conf. Emerg. Netw. Experiments Technol.*, 2013, pp. 283–294.
- [48] S. Eismann et al., "Serverless applications: Why, when, and how?," *IEEE Softw.*, vol. 38, no. 1, pp. 32–39, Jan./Feb. 2021.
- [49] J. Dean and L. A. Barroso, "The tail at scale," *Commun. ACM*, vol. 56, pp. 74–80, 2013.
- [50] F. C. Freiling et al., "The failure detector abstraction," *ACM Comput. Surv.*, vol. 43, no. 2, pp. 9:1–9:40, 2011.
- [51] M. Shahrad et al., "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *Proc. USENIX Annu. Tech. Conf.*, 2020, pp. 205–218.
- [52] H. S. Gunawi et al., "Why does the cloud stop computing? lessons from hundreds of service outages," in *Proc. 17th ACM Symp. Cloud Comput.*, 2016, pp. 1–16.
- [53] S. Gupta, T. Patel, C. Engelmann, and D. Tiwari, "Failures in large scale systems: Long-term measurement, analysis, and implications," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2017, pp. 44:1–44:12.
- [54] R. Birke, I. Giurgiu, L. Y. Chen, D. Wiesmann, and T. Engbersen, "Failure analysis of virtual and physical machines: Patterns, causes and characteristics," in *Proc. 44th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2014, pp. 1–12.
- [55] A. Taherin, T. Patel, G. Georgakoudis, I. Laguna, and D. Tiwari, "Examining failures and repairs on supercomputers with multi-GPU compute nodes," in *Proc. 51st Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2021, pp. 305–313.
- [56] R. Kumar, "The mystery of the failing jobs: Insights from operational data from two university-wide computing systems," in *Proc. 50th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2020, pp. 158–171.
- [57] Y. Chen, K. Srinivasan, G. R. Goodson, and R. H. Katz, "Design implications for enterprise storage systems via multi-dimensional trace analysis," in *Proc. 23rd ACM Symp. Operating Syst. Princ.*, 2011, pp. 43–56.
- [58] S. Kavulya, J. Tan, R. Gandhi, and P. Narasimhan, "An analysis of traces from a production mapreduce cluster," in *Proc. 10th IEEE/ACM Int. Conf. Cluster, Cloud Grid Comput.*, 2010, pp. 94–103.
- [59] Z. Zheng et al., "Co-analysis of RAS log and job log on blue gene/p," in *Proc. 25th IEEE Int. Symp. Parallel Distrib. Process.*, 2011, pp. 840–851.
- [60] N. El-Sayed and B. Schroeder, "Reading between the lines of failure logs: Understanding how HPC systems fail," in *Proc. 43rd Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2013, pp. 1–12.
- [61] C. D. Martino, Z. T. Kalbarczyk, R. K. Iyer, F. Baccanico, J. Fullop, and W. Kramer, "Lessons learned from the analysis of system failures at petascale: The case of blue waters," in *Proc. 44th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2014, pp. 610–621.
- [62] S. Di, H. Guo, E. Pershey, M. Snir, and F. Cappello, "Characterizing and understanding HPC job failures over the 2k-day life of IBM bluegene/q system," in *Proc. 49th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2019, pp. 473–484.
- [63] D. Yuan et al., "Simple testing can prevent most critical failures: An analysis of production failures in distributed data-intensive systems," in *Proc. 11th USENIX Symp. Operating Syst. Des. Implementation*, 2014, pp. 249–265.
- [64] Z. Yin, X. Ma, J. Zheng, Y. Zhou, L. N. Bairavasundaram, and S. Pasupathy, "An empirical study on configuration errors in commercial and open source systems," in *Proc. 23rd ACM Symp. Operating Syst. Princ.*, 2011, pp. 159–172.
- [65] P. Fonseca, C. Li, V. Singhal, and R. Rodrigues, "A study of the internal and external effects of concurrency bugs," in *Proc. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2010, pp. 221–230.
- [66] F. Frattini, R. Ghosh, M. Cinque, A. Rindos, and K. S. Trivedi, "Analysis of bugs in apache virtual computing lab," in *Proc. 43rd Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2013, pp. 1–6.
- [67] B. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," *Evidence-Based Softw. Eng.*, Tech. Rep. EBSE-2007-01, 2007.
- [68] S. Talluri, L. Overweel, L. Versluis, A. Trivedi, and A. Iosup, "Empirical characterization of user reports about cloud failures," in *Proc. IEEE Int. Conf. Autonomic Comput. Self-Organizing Syst.*, E. El-Araby, V. Kalogeraki, D. Pianini, F. Lassabe, B. Porter, S. Ghahremani, I. Nunes, M. Bakhouya, and S. Tomforde, Eds., IEEE, 2021, pp. 158–163, doi: [10.1109/ACSOS52086.2021.00039](https://doi.org/10.1109/ACSOS52086.2021.00039).
- [69] S. Ghosh, M. Shetty, C. Bansal, and S. Nath, "How to fight production incidents?: An empirical study on a large-scale cloud service," in *Proc. 13th Symp. Cloud Comput.*, A. Gavrilovska, D. Altinbükten, and C. Binnig, Eds., ACM, 2022, pp. 126–141, doi: [10.1145/3542929.3563482](https://doi.org/10.1145/3542929.3563482).
- [70] D. Cotroneo, L. D. Simone, P. Liguori, R. Natella, and N. Bidokhti, "How bad can a bug get? an empirical analysis of software failures in the openstack cloud computing platform," in *Proc. ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Foundations Softw. Eng.*, M. Dumas, D. Pfahl, S. Apel, and A. Russo, Eds., ACM, 2019, pp. 200–211, doi: [10.1145/3338906.3338916](https://doi.org/10.1145/3338906.3338916).
- [71] S. Burnett et al., "Network error logging: Client-side measurement of end-to-end web service reliability," in *Proc. 17th USENIX Symp. Networked Syst. Des. Implementation*, 2020, pp. 985–998.
- [72] G. Li et al., "Understanding error propagation in deep learning neural network (DNN) accelerators and applications," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, B. Mohr and P. Raghavan, Eds., ACM, 2017, pp. 1–12, doi: [10.1145/3126908.3126964](https://doi.org/10.1145/3126908.3126964).
- [73] V. Oles, A. Schmedding, G. Ostrouchov, W. Shin, E. Smirni, and C. Engelmann, "Understanding GPU memory corruption at extreme scale: The summit case study," in *Proc. 38th ACM Int. Conf. Supercomputing*, K. Kise, V. Salapura, M. Annavaram, and A. L. Varbanescu, Eds., ACM, 2024, pp. 188–200, doi: [10.1145/3650200.3656615](https://doi.org/10.1145/3650200.3656615).
- [74] D. Tiwari, S. Gupta, G. Gallarno, J. Rogers, and D. Maxwell, "Reliability lessons learned from GPU experience with the titan supercomputer at oak ridge leadership computing facility," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, J. Kern and J. S. Vetter, Eds., ACM, 2015, pp. 38:1–38:12, doi: [10.1145/2807591.2807666](https://doi.org/10.1145/2807591.2807666).
- [75] G. Ostrouchov, D. Maxwell, R. A. Ashraf, C. Engelmann, M. Shankar, and J. H. Rogers, "GPU lifetimes on titan supercomputer: Survival analysis and reliability," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, C. Cuicchi, I. Qualters, and W. T. Kramer, Eds., IEEE/ACM, 2020, pp. 1–14, doi: [10.1109/SC41405.2020.00045](https://doi.org/10.1109/SC41405.2020.00045).
- [76] S. Cui et al., "Story of Two GPUs: Characterizing the Resilience of Hopper H100 and Ampere A100 GPUs," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2025, pp. 1145–1164.
- [77] B. Schroeder, R. Lagisetty, and A. Merchant, "Flash reliability in production: The expected and the unexpected," in *Proc. 14th USENIX Conf. File Storage Technol.*, A. D. Brown and F. I. Popovici, Eds., USENIX Association, 2016, pp. 67–80. [Online]. Available: <https://www.usenix.org/conference/fast16/technical-sessions/presentation/schroeder>
- [78] N. R. Mielke, R. E. Frickey, I. Kalastirsky, M. Quan, D. Ustinov, and V. J. Vasudevan, "Reliability of solid-state drives based on NAND flash memory," *Proc. IEEE*, vol. 105, no. 9, pp. 1725–1750, Sep. 2017, doi: [10.1109/JPROC.2017.2725738](https://doi.org/10.1109/JPROC.2017.2725738).
- [79] Y. Cai, S. Ghose, E. F. Haratsch, Y. Luo, and O. Mutlu, "Error characterization, mitigation, and recovery in flash-memory-based solid-state drives," *Proc. IEEE*, vol. 105, no. 9, pp. 1666–1704, Sep. 2017, doi: [10.1109/JPROC.2017.2713127](https://doi.org/10.1109/JPROC.2017.2713127).
- [80] R. Natella, D. Cotroneo, and H. Madeira, "Assessing dependability with software fault injection: A survey," *ACM Comput. Surv.*, vol. 48, no. 3, pp. 44:1–44:55, 2016, doi: [10.1145/2841425](https://doi.org/10.1145/2841425).
- [81] L. Guo, G. Georgakoudis, K. Parasyris, I. Laguna, and D. Li, "MATCH: An MPI fault tolerance benchmark suite," in *Proc. IEEE Int. Symp. Workload Characterization*, 2020, pp. 60–71, doi: [10.1109/IISWC50251.2020.00015](https://doi.org/10.1109/IISWC50251.2020.00015).
- [82] K. Kingsbury et al., "Jepsen: A framework for distributed systems verification, with fault injection." 2025. [Online]. Available: <https://github.com/jepsen-io/jepsen>

- [83] T. Lyu et al., "Monarch: A fuzzing framework for distributed file systems," in *Proc. USENIX Annu. Techn. Conf.*, S. Bagchi and Y. Zhang, Eds., USENIX Association, 2024, pp. 529–543. [Online]. Available: <https://www.usenix.org/conference/atc24/presentation/lyu>
- [84] D. G. Feitelson, *Workload Modeling for Computer Systems Performance Evaluation*. Cambridge, U.K.: Cambridge Univ. Press, 2015.
- [85] SNIA, "Storage networking industry association's input/output traces, tools, and analysis," 2024. [Online]. Available: <https://iota.snia.org/>
- [86] S. Samsi et al., "The MIT supercloud dataset," in *Proc. IEEE High Perform. Extreme Comput. Conf.*, 2021, pp. 1–8.
- [87] J. Yang, Y. Yue, and K. V. Rashmi, "A large-scale analysis of hundreds of in-memory key-value cache clusters at twitter," *ACM Trans. Storage*, vol. 17, no. 3, pp. 17:1–17:35, 2021.
- [88] A. Borghesi et al., "M100 exadata: A data collection campaign on the cineca's marconi100 tier-0 supercomputer," *Sci. Data*, vol. 10, no. 1, 2023, Art. no. 288.
- [89] X. Chu et al., "Generic and ML workloads in an HPC datacenter: Node energy, job failures, and node-job analysis," in *Proc. 30th IEEE Int. Conf. Parallel Distrib. Syst.*, 2024, pp. 710–719, doi: [10.1109/ICPADS63350.2024.00097](https://doi.org/10.1109/ICPADS63350.2024.00097).
- [90] J. Wilkes, "Yet more Google compute cluster trace data," in *Google Research Blog*, Mountain View, CA, USA, Apr. 2020. [Online]. Available: <https://ai.googleblog.com/2020/04/yet-more-google-compute-cluster-trace.html>
- [91] O. Hadary et al., "Protean: VM allocation service at scale," in *Proc. 14th USENIX Symp. Operating Syst. Des. Implementation, OSDI 2020, Virtual Event*, 2020, pp. 845–861.
- [92] Q. Weng et al., "Mlaas in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters," in *Proc. 19th USENIX Symp. Networked Syst. Des. Implementation*, 2022, pp. 945–960.
- [93] A. Uta et al., "Is Big Data performance reproducible in modern cloud networks?," in *Proc. 17th USENIX Symp. Netw. Syst. Des. Implementation*, R. Bhagwan and G. Porter, Eds., USENIX Association, 2020, pp. 513–527. [Online]. Available: <https://www.usenix.org/conference/nsdi20/presentation/uta>
- [94] D. D. Sensi, T. D. Matteis, K. Taranov, S. D. Girolamo, T. Rahn, and T. Hoefler, "Noise in the clouds: Influence of network performance variability on application scalability," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 6, no. 3, pp. 49:1–49:27, 2021, doi: [10.1145/3570609](https://doi.org/10.1145/3570609).
- [95] J. Freischuetz, K. Kanellis, B. Kroth, and S. Venkataraman, "TUNA: Tuning unstable and noisy cloud applications," in *Proc. 20th Eur. Conf. Comput. Syst.*, 2025, pp. 954–973, doi: [10.1145/3689031.3717480](https://doi.org/10.1145/3689031.3717480).



Xiaoyu Chu is currently working toward the PhD degree with Vrije Universiteit Amsterdam. Her work characterizes the reliability of HPC datacenters and popular ML-focused cloud services.



Jakob Kyselica received the MSc degree from Vrije Universiteit Amsterdam focusing on the reliability of online games. He is currently a software engineer with MEOS.



Mehmet Cetin received the MSc degree from Vrije Universiteit Amsterdam focusing on the reliability of public cloud services. He is currently a software engineer with Amazon.



Alexander Balgavy received the MSc degree from Vrije Universiteit Amsterdam, focusing on vulnerability analysis of non-Linux firmware. He is currently a software engineer, currently building software for the International Space Station with Space Applications Services.



Sacheendra Talluri received the PhD degree from Vrije Universiteit Amsterdam. He is currently a senior software engineer with ClickHouse. His work focuses on autoscaling, fault tolerance in the cloud and the structure of cloud resource managers.



Dante Niewenhuis is currently working toward the PhD degree with Vrije Universiteit Amsterdam. His work focuses on using simulation and digital twinning to improve the sustainability of datacenters.



Alexandru Iosup (Member, IEEE) is currently a full professor with Vrije Universiteit Amsterdam and leads the AtLarge research group. He has been awarded the Netherlands ICT-Researcher of the Year award, member of the Royal Netherlands Young Academy of Arts and Sciences, and has been knighted by the Romanian Government.